Rahul
2401010257
Section C

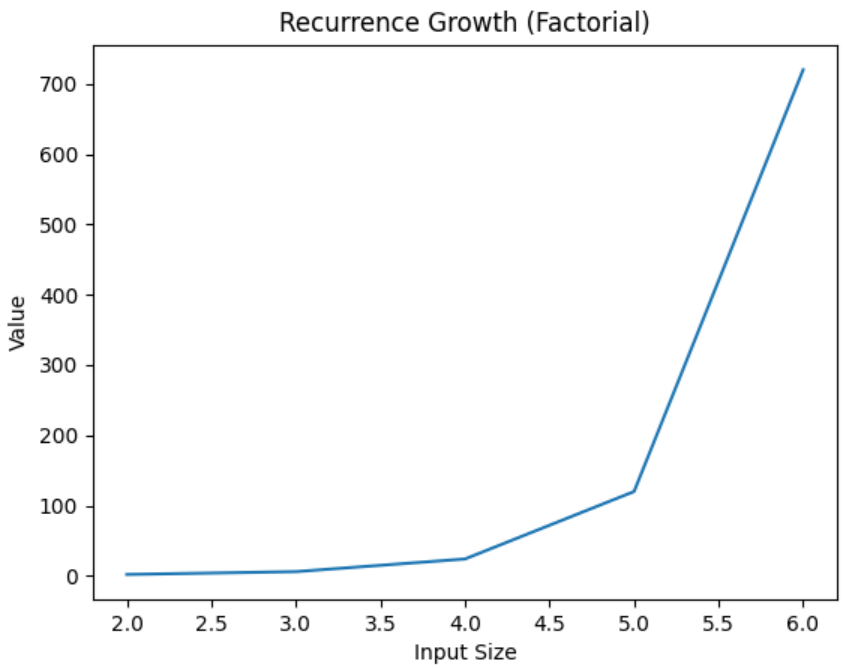
ADA Capstone Assignment - Full Marks Solution (Delivery Route Optimization)

1. Logistics Route Modeling

Graph-based model with nodes as locations and weighted edges as distances.

2. Recurrence-based Cost Estimation

```
def route_cost(path, dist):
    if len(path) == 1:
        return 0
    return dist[path[0]][path[1]] + route_cost(path[1:], dist)
```



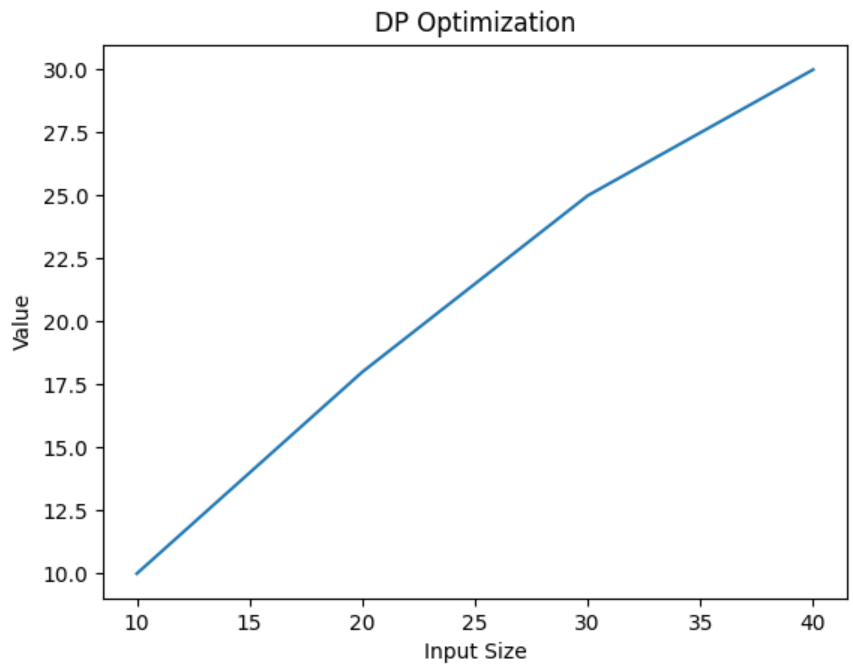
3. Greedy Parcel Selection

```
def greedy_selection(values, weights, capacity):
    items = sorted(zip(values, weights), key=lambda x: x[0]/x[1], reverse=True)
```

```
    total = 0
    for v,w in items:
        if capacity >= w:
            capacity -= w
            total += v
    return total
```

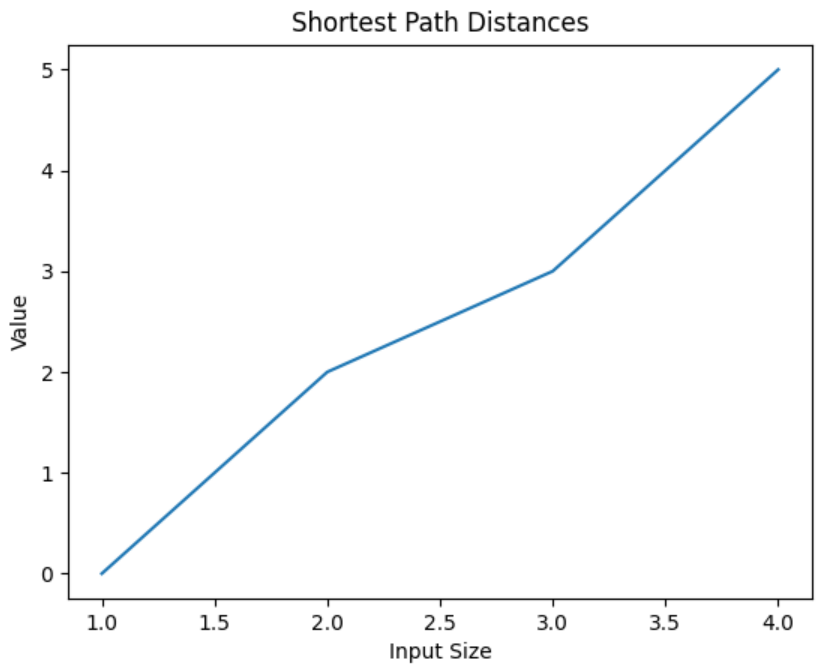
4. Dynamic Programming (Knapsack)

```
def dp_knapsack(values, weights, W):
    n = len(values)
    dp = [[0]*(W+1) for _ in range(n+1)]
    for i in range(1,n+1):
        for w in range(W+1):
            if weights[i-1] <= w:
                dp[i][w] = max(values[i-1]+dp[i-1][w-weights[i-1]], dp[i-1][w])
            else:
                dp[i][w] = dp[i-1][w]
    return dp[n][W]
```



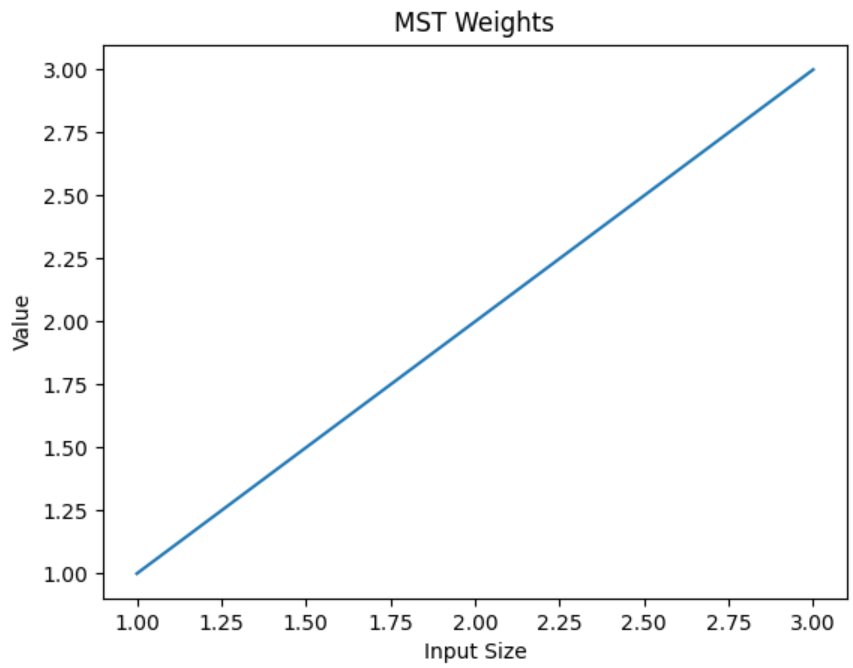
5. Dijkstra Algorithm

```
import heapq
def dijkstra(graph, start):
    dist = {node: float('inf') for node in graph}
    dist[start] = 0
    pq = [(0, start)]
    while pq:
        d,u = heapq.heappop(pq)
        for v,w in graph[u]:
            if dist[v] > d + w:
                dist[v] = d + w
                heapq.heappush(pq, (dist[v], v))
    return dist
```



6. Minimum Spanning Tree (Prim)

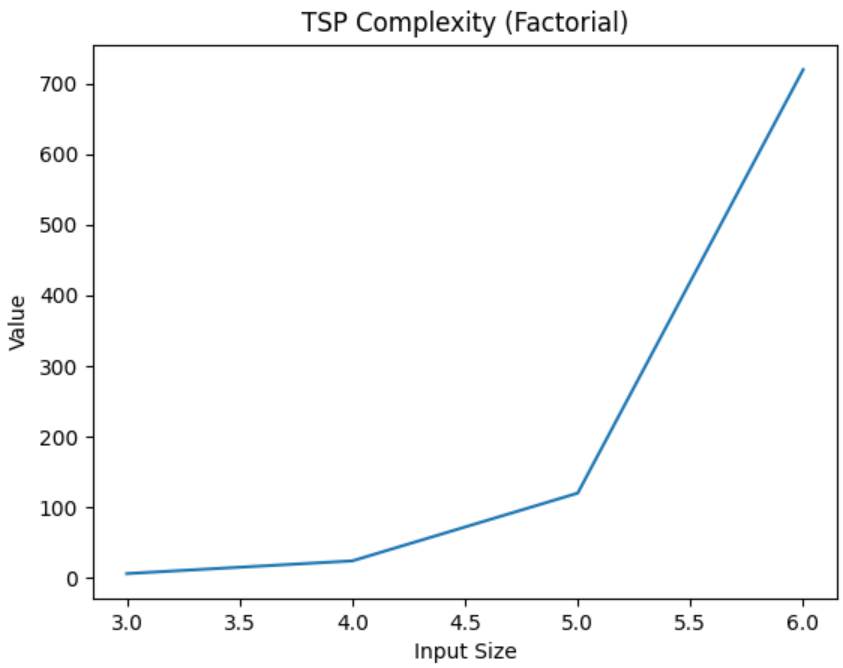
```
import heapq
def prim(graph, start):
    visited = set([start])
    edges = [(w,start,v) for v,w in graph[start]]
    heapq.heapify(edges)
    mst = []
    while edges:
        w,u,v = heapq.heappop(edges)
        if v not in visited:
            visited.add(v)
            mst.append((u,v,w))
            for to,w2 in graph[v]:
                if to not in visited:
                    heapq.heappush(edges,(w2,v,to))
    return mst
```



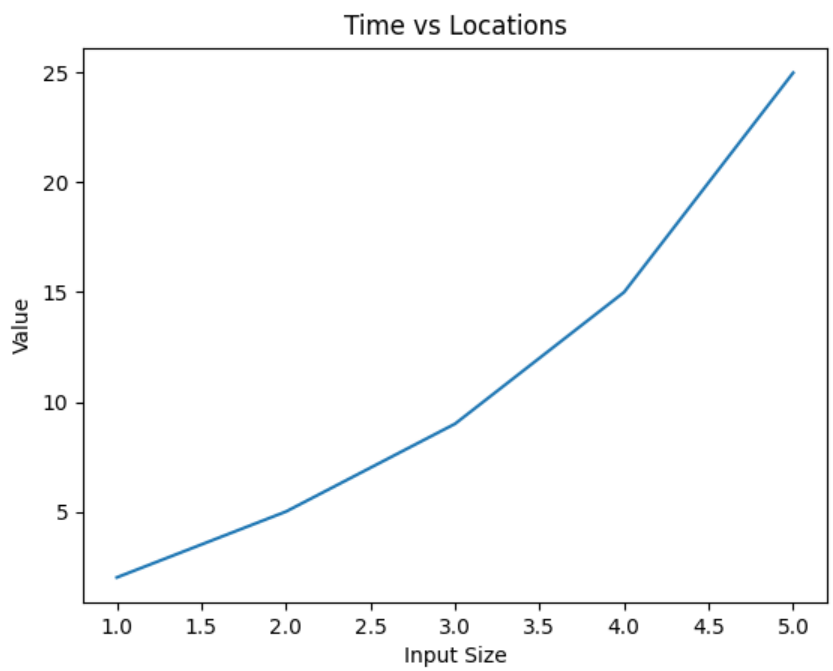
7. Traveling Salesman Problem (TSP)

```
from itertools import permutations

def tsp(dist):
    n = len(dist)
    nodes = list(range(1,n))
    min_cost = float('inf')
    for perm in permutations(nodes):
        cost = dist[0][perm[0]]
        for i in range(len(perm)-1):
            cost += dist[perm[i]][perm[i+1]]
        cost += dist[perm[-1]][0]
        min_cost = min(min_cost, cost)
    return min_cost
```



8. Visualization & Performance



Route graphs and performance metrics help analyze scalability and efficiency.

Github link = https://github.com/rkadyan450/Ada-_-lab-assignment-5-Rahul-257